

帮助文档 1

——如何构建硬件系统

电子系统专题设计与制作

2026 年 6 月 30 日

适用对象

选修《电子系统专题设计与制作》的同学

前置阅读

大作业方案

建议用时

动手实现前，先花 1 小时阅读并完成本文档中的分析练习

目录

引言 3

1 自顶向下构建系统 3

1.1 什么是“自顶向下”？ 3

1.2 自顶向下的四个层次 3

1.3 为什么要自顶向下？ 3

2 明确功能约束 4

2.1 第一步：仔细阅读大作业方案 4

2.2 第二步：从方案中提取关键信息 4

2.2.1 系统有哪些输入？ 4

2.2.2 系统有哪些输出？ 4

2.2.3 系统有哪些状态变量？ 5

2.2.4 系统有哪些时序约束？ 5

2.3 第三步：整理成需求规格表 6

3 硬件架构设计 6

3.1 认识你的核心器件 6

3.2 从功能到外设：接口类型分析 7

3.3 引脚规划 8

3.3.1 需要查阅数据手册的引脚限制 8

3.3.2 引脚分配的原则 8

3.4 系统框图 9

3.4.1 什么是系统框图？ 9

3.4.2 框图的基本元素 9

3.4.3 绘制步骤 10

3.4.4 画图工具 10

3.4.5 常见错误 11

3.4.6 引导问题 11

3.5 电源与电平匹配 11

4 软件架构设计	12
4.1 软件分层	12
4.2 程序调度方式	12
4.2.1 方式一：超级循环（Super Loop）	12
4.2.2 方式二：定时器中断 + 超级循环	13
4.2.3 方式三：RTOS 多任务	13
4.3 状态机设计	13
4.3.1 如何画状态转换图	14
4.4 模块间通信与耦合	15
4.5 定时与计时	15
4.6 软件流程图：从思路到图形	15
4.6.1 什么是软件流程图？	16
4.6.2 流程图的基本符号	16
4.6.3 你需要画哪些流程图？	16
4.6.4 流程图绘制步骤	18
4.6.5 画图工具	19
4.6.6 常见错误	19
4.6.7 引导问题	20
5 从设计到实现：下一步行动	20
附录附加资源	20

引言

此文档旨在帮助同学们完成《电子系统专题设计与制作》的大作业。阅读本文档时，请始终记住：**本文档的目标是引导你独立思考系统的架构设计，而不是直接告诉你答案。**大作业的核心价值在于你亲自完成从需求分析到系统实现的全过程。当你面对一个设计决策时，先想一想“为什么”，再动手做。

1 自顶向下构建系统

1.1 什么是“自顶向下”？

想象你要建造一栋房子。你不会一上来就去挑选每一块砖头——你会先画建筑蓝图，确定有几个房间、每个房间的功能、房间之间的连通关系，最后才去关心具体的建材和施工细节。

嵌入式系统设计也是如此。**自顶向下**（Top-Down）的设计方法要求你从最高层次的用户需求出发，逐层向下细化，直到具体的硬件连线 and 软件代码。它的反面是“自底向上”——先摆弄各个模块，再试图拼在一起，往往会导致模块之间无法协调工作。

1.2 自顶向下的四个层次

层次	关注的问题	本阶段的输出
需求分析	系统要做什么？用户如何与之交互？	功能列表、约束条件清单
架构设计	系统由哪些模块组成？模块之间如何通信？	系统框图、模块接口定义
详细设计	每个模块如何实现？用什么外设？什么算法？	原理图、引脚分配表、软件流程图
实现与验证	逐个模块编写代码、连接硬件、调试	可运行的代码、调试记录

本帮助文档聚焦于前两个层次——**需求分析和架构设计**。这两个层次决定了你的系统是“设计出来的”还是“凑出来的”。

1.3 为什么要自顶向下？

- **避免返工**：先想清楚再动手，比边做边改效率高得多。
- **模块化解耦**：清晰的模块边界让你可以独立调试每个部分。
- **可扩展性**：好的架构让你的扩展功能更容易加入，而不是在已有代码上打补丁。

想一想

如果你直接开始焊接电路、写代码，而没有一个整体的架构规划，当你在验收前三天发现两个功能模块之间存在冲突时，你还有多少时间可以修改？

2 明确功能约束

在开始任何设计之前，你首先需要完整、准确地理解系统需要做什么。遗漏一个功能需求，可能意味着你需要重新设计一半的硬件和软件。

2.1 第一步：仔细阅读大作业方案

大作业方案是你最根本的设计依据。建议你至少通读两遍：

1. **第一遍**：了解整体框架，知道桌宠大致需要哪些功能。
2. **第二遍**：逐条记录每个功能的细节，特别注意带有数字的约束条件（时间、概率、数值范围等）。

2.2 第二步：从方案中提取关键信息

请尝试从大作业方案中提取以下信息。下面给出了一些引导性问题，**请你在阅读方案后自己回答这些问题**，而不是直接看后面的分析。

2.2.1 系统有哪些输入？

思考：用户通过什么方式与桌宠交互？环境信息如何被桌宠感知？电脑端能发送什么指令？

引导问题

列出所有输入来源后，再标注每种输入属于什么类型——是数字信号（高低电平）还是模拟信号（连续电压）？是脉冲序列还是数据帧？

参考清单（建议先自行列出后再看）

- 按钮——数字输入，按下/释放两种状态
- 编码器——脉冲序列，需要检测旋转方向和步数
- 光敏电阻——模拟信号，光照强度变化引起电压变化
- 串口指令——数字数据帧，来自电脑的文本指令

2.2.2 系统有哪些输出？

思考：桌宠如何向用户展示自己的状态？如何执行物理动作？如何与电脑通信？

引导问题

不同类型的输出可能需要完全不同的驱动方式。例如，显示文字需要通信协议，驱动舵机需要特定波形的信号。

参考清单（建议先自行列出后再看）

- OLED 屏幕——显示文本信息（心情值、饱食状态、当前动作）
- 舵机 $\times 2$ ——执行物理动作，需要精确的角度控制
- 串口调试信息——向电脑发送状态数据（文本格式）

2.2.3 系统有哪些状态变量？

思考：桌宠的“内心世界”由哪些变量构成？这些变量的取值范围是什么？它们之间有什么关系？

引导问题

状态变量不是硬件，而是存在软件中的“记忆”。但它们会影响你对定时器、中断等硬件资源的需求。

参考清单（建议先自行列出后再看）

- **心情值**：1 ~ 10 的整数，初始为 5，影响行为表现
- **饱食状态**：饥饿 / 吃饱（二值状态），影响心情值是否衰减
- **饱食剩余时间**：吃饱后 2 分钟倒计时
- **当前动作**：无动作 / 玩耍 / 闲逛 / 疲惫 / 前进（5 种状态之一）
- **动作持续时间**：每种动作有指定的持续时间（2 ~ 4 秒）
- **睡眠状态**：清醒 / 睡眠（二值状态），影响几乎所有其他行为
- **摸头冷却**：10 秒内不能重复摸头
- **低光照连续时间**：连续 5 秒判定进入睡眠

2.2.4 系统有哪些时序约束？

思考：大作业方案中提到了哪些时间相关的规则？把它们全部列出来。

引导问题

时序约束是嵌入式系统中最容易被忽视也最容易出 Bug 的地方。每个时间约束都意味着你需要某种计时机制。

参考清单（建议先自行列出后再看）

时序规则	时间参数
心情值自然衰减	每 30 秒下降 1 点
吃饱转饥饿	进入吃饱后 2 分钟
自动动作触发	无动作时每秒 3.33% 概率检查
摸头冷却	摸头后 10 秒内不可再次触发
低光照判定	连续 5 秒低光照进入睡眠
状态信息发送	每 5 秒发送一次
各动作持续时间	2 ~ 4 秒（见动作表）

2.3 第三步：整理成需求规格表

把你提取出的所有信息整理成一张表格。这不是形式主义——当你开始设计时，你会频繁回来看这张表，确认自己没有遗漏。

重要

这里不提供完整的表格模板，因为整理的过程本身就是设计的一部分。建议你按“输入 / 输出 / 状态 / 时序 / 通信”五个类别来组织。

3 硬件架构设计

当你明确了系统需要做什么之后，下一步是思考用什么硬件资源来支撑这些功能。

3.1 认识你的核心器件

大作业指定使用 ESP32-S3-WROOM-1-N16R8 模组作为主控制器。在做任何设计决策之前，你应当了解这颗芯片提供了哪些资源。下面是你在设计时需要关注的关键参数：

资源	规格	对你的意义
处理器	Xtensa LX7 双核 @ 240 MHz	有足够的算力处理复杂任务
Flash	16 MB	程序存储空间充裕, 可考虑存储数据
PSRAM	8 MB	运行内存充裕
GPIO	约 36 个可用	需合理分配, 注意复用冲突
ADC	2 个 12-bit SAR ADC (共约 20 通道)	用于读取模拟量 (如光敏电阻)
LEDC (PWM)	8 通道	用于驱动舵机
I2C	2 组	可用于连接 OLED 屏幕
UART	3 组	用于烧录程序、串口通信和调试
GPIO 中断	所有 GPIO 支持	用于按钮和编码器输入
定时器	多个 54-bit 通用定时器	是实现各种时序约束的基础

重要

ESP32-S3 的很多 GPIO 具有多个可选功能 (通过 GPIO Matrix 路由), 但某些外设 (如 ADC) 只能在特定引脚上使用。查阅数据手册是必不可少的步骤。

3.2 从功能到外设：接口类型分析

现在请做下面的练习：将你的每一项硬件需求映射到 ESP32-S3 的某一种外设资源。

硬件需求	需要的接口类型	你计划使用的 ESP32-S3 外设
OLED 屏幕	? (串行通信协议)	?
舵机 × 2	? (需要特定波形)	?
光敏电阻	? (模拟电压读取)	?
按钮	? (数字电平检测)	?
编码器	? (脉冲信号检测)	?
串口通信	? (异步串行通信)	?

引导问题

1. OLED 屏幕支持的通信协议通常有哪几种？查阅你手头 OLED 模块的数据手册，确认它使用哪种协议，以及协议需要几根信号线。
2. 舵机是如何通过电信号控制角度的？查阅舵机的工作原理（关键词：PWM、占空比、周期）。
3. 光敏电阻为什么需要 ADC？分压电路应该如何设计？
4. 编码器的 A/B 相信号有什么特征？为什么用普通 GPIO 读取可能会丢脉冲？什么情况下需要用到中断？

重要

本文档不会替你填写这个表格——这需要你查阅各个器件的数据手册来完成。在大作业设计报告中，“查阅数据手册获取的关键参数”是一个重要的评分点。

3.3 引脚规划

ESP32-S3 虽然有约 36 个可用 GPIO，但并非每个引脚都“平等”。在分配引脚时，你需要特别注意以下约束：

3.3.1 需要查阅数据手册的引脚限制

以下内容是你在引脚分配时必须关注的问题。请带着这些问题去查阅 ESP32-S3-WROOM-1 的数据手册：

1. **Strapping 引脚**：某些引脚在上电时会被芯片采样以决定启动模式。如果这些引脚被你用作其他功能，外部电路是否会干扰启动过程？
2. **ADC 可用引脚**：并非所有 GPIO 都可以用作 ADC 输入。ADC1 和 ADC2 分别连接在哪些 GPIO 上？ADC2 有什么使用限制（提示：与 Wi-Fi 有关）？
3. **Flash/PSRAM 占用**：WROOM-1 模组内部已经连接了 Flash 和 PSRAM 的引脚。N16R8 使用 Octal PSRAM，这会占用哪些 GPIO？这些 GPIO 你还能用作其他用途吗？
4. **烧录引脚**：内置 UART 烧录端口默认使用哪些引脚？如果你用这些引脚连接了其他外设，是否会影响程序下载和调试？

引导问题

在分配引脚之前，请先在数据手册中找到“Pin Layout”和“Strapping”相关章节，标出所有具有特殊功能的引脚。然后思考：你需要的 GPIO 数量够吗？有没有引脚功能冲突？

3.3.2 引脚分配的原则

当你开始具体分配引脚时，可以参考以下原则：

- **先分配“挑剔”的外设**：有些外设只能在特定引脚上工作（如 ADC），先给它们分配引脚，再分配“灵活”的外设（如 GPIO 输入输出）。
- **预留调试接口**：至少保留 UART0（或 USB Serial/JTAG）用于下载程序和输出调试信息。
- **考虑 PCB 布线**：如果你计划自行设计 PCB，相邻的 I2C 引脚、PWM 引脚尽量靠近，减少走线交叉。

- **为扩展功能留有余地：**如果你计划添加蓝牙、Wi-Fi、更多传感器等扩展功能，确保相关引脚不会被基础功能占用。

重要

本文档不会给你一个现成的引脚分配表。在大作业中，引脚分配是你需要独立完成的设计决策，并且需要在设计报告中说明你的选型依据。

3.4 系统框图

3.4.1 什么是系统框图？

系统框图（Block Diagram）是用图形化的方式展示硬件系统由哪些模块组成、模块之间如何连接。它关注的是“谁和谁相连、通过什么接口相连”，而不关心具体的引脚编号或芯片内部细节。

可以把系统框图理解为硬件系统的“地图”——看到它，读者就能立刻理解整个系统的组成结构和信号流动关系。在设计报告中，系统框图是必画图，也是助教理解你设计思路的第一入口。

3.4.2 框图的基本元素

系统框图中常用的元素非常简单，只有三种：

元素	画法	用途
模块框	矩形方框，内部写模块名称	表示一个硬件模块（MCU、传感器、舵机等）
连线	直线（可带箭头）	表示模块之间的电气连接
标注	文字标注在连线旁	说明该连接的接口类型或信号含义（如 I2C、PWM、ADC）

下面是一个简单的示例（图1），展示三个模块（MCU、OLED 屏幕、舵机）的连接关系：

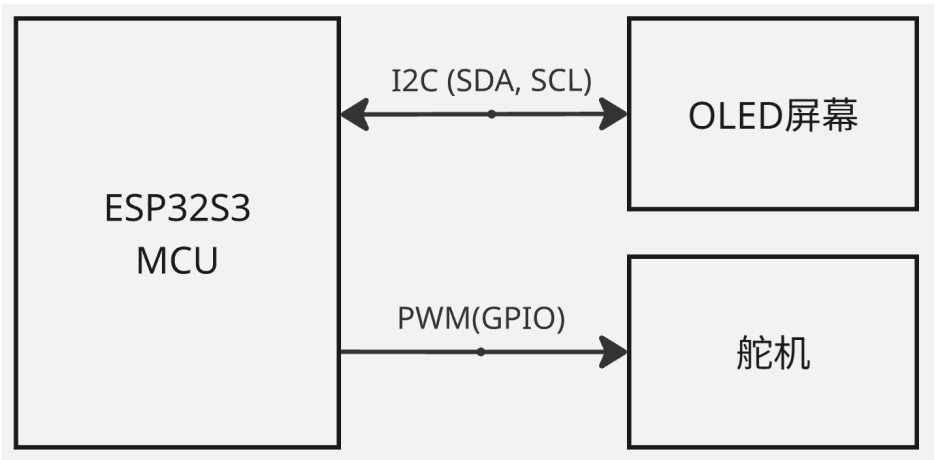


图 1: 系统框图示例

引导问题

上图中，I2C 连线是双向的（OLED 的显示数据从 MCU 发出，但 I2C 协议本身需要 ACK 应答），所以用双向箭头。PWM 信号只从 MCU 发出到舵机，所以用单向箭头。箭头方向代表信号的流向，这有助于理解谁控制谁。

3.4.3 绘制步骤

如果你是第一次画系统框图，可以按照以下步骤来：

第 1 步：列出所有硬件模块

先把你的系统中所有的硬件组件列一个清单。以智能桌宠为例，你至少应该包括：

- MCU 核心板（ESP32-S3-WROOM-1）
- 各个输入设备（按钮、编码器、光敏电阻模块）
- 各个输出设备（OLED 屏幕、舵机 × 2）
- 通信接口（USB 转串口，连接电脑）
- 电源部分（电池 / USB 供电 / 稳压模块）

想一想

把每个模块写在一张便利贴上，这样可以随意移动排列。

第 2 步：确定核心，围绕它布局

MCU 是系统的核心，通常放在图的正中央。输入设备放在左侧，输出设备放在右侧，这是一种常见且直观的布局习惯（信号从左流向右）。

第 3 步：画出连接线，标注接口类型

逐一画出 MCU 到每个外设的连线，并在连线旁标注：

- 接口协议（I2C、SPI、UART、PWM、ADC、GPIO）
- 关键信号名（如 SDA、SCL、PWM1、ADC_IN 等）
- 如果是多根线，可以用一根粗线表示总线，旁边注明 “×N”

第 4 步：补充电源和调试接口

这是初学者最容易遗漏的部分：

- 电源是如何供给的？各模块分别用多少 V？是否需要独立的稳压？
- 程序下载和调试用什么接口？（USB Serial/JTAG、UART0 等）
- 如果有电平转换、驱动电路（如舵机驱动），也应在图中体现。

第 5 步：检查与修正

画完后，对照你的功能需求清单逐条检查：每个硬件需求是否都在框图中有对应的模块？每条连线是否都标注了接口类型？

3.4.4 画图工具

以下工具都可以用来画系统框图，按上手难度排序：

工具	特点	推荐度
纸 + 笔	最快、最灵活，适合初稿和思路整理	初稿首选
Draw.io (draw.io / diagrams.net)	免费、浏览器直接使用、元件库丰富、支持导出 PNG/SVG	强烈推荐
PowerPoint / Keynote	人人都有，画简单框图够用	可用
Visio	功能全面，但需要授权	可选
Figma	在线协作，美观	可选
KiCad / Eeschema	原理图工具，可直接转 PCB	如要做 PCB 推荐

想一想

建议流程：先用纸笔打草稿 → 确定布局后在 Draw.io 中画正式版 → 导出图片插入设计报告。

3.4.5 常见错误

- **连线没有标注：**光秃秃一根线，读者不知道是什么协议、什么信号。每条重要连线都应该标注接口类型。
- **把框图当成引脚接线图：**不需要标注 GPIO 编号（GPIO4、GPIO5 等），那是原理图或引脚分配表的事情。框图关注的是“接口类型”而非“具体引脚”。
- **遗漏电源和调试接口：**“电源怎么接的”和“程序怎么下载的”是初学者经常忘记画的部分。
- **模块框太笼统：**把“传感器”写成一个框，但实际你有光敏电阻、编码器等不同的传感器，它们与 MCU 的接口类型完全不同，应该分开画。
- **缺少信号方向：**箭头方向表示信号的流向（谁发送、谁接收），这对理解系统很重要。

3.4.6 引导问题

画出第一版框图之后，请检查：

- 图中的每一条连线，你是否清楚地知道它代表什么信号？信号的电气特性是什么（电压、频率、协议）？
- 有没有漏掉的模块？（提示：电源怎么来的？下载调试怎么连接？）
- 各模块之间的耦合关系是否合理？如果你需要更换其中一个模块（比如换一个不同型号的 OLED），需要改动哪些部分？

3.5 电源与电平匹配

嵌入式系统设计中，电源和电平匹配是最容易被忽视但最致命的问题。

引导问题

- 1. ESP32-S3 的 GPIO 工作电压是多少？你的各个外设（OLED、舵机、光敏模块）的工作电压分别是多少？查阅它们的数据手册确认。
- 2. 如果外设的工作电压与 MCU 不同，你应该怎么做？（提示：电平转换）
- 3. 舵机通常需要较大的驱动电流。ESP32-S3 的 GPIO 能直接驱动舵机吗？如果不能，你需要额外的什么电路？
- 4. 整个系统需要几种电压轨？（提示：MCU 需要 3.3 V，舵机可能需要 5 V，还有其他的吗？）你打算如何提供这些电压？

4 软件架构设计

硬件架构关注的是“信号怎么流动”，而软件架构关注的是“数据怎么处理”和“任务怎么调度”。

4.1 软件分层

良好的嵌入式软件通常采用分层架构（见图2）。每一层只依赖它下面的层，而不依赖它上面的层：

应用层（Application）	业务逻辑：心情管理、动作决策、状态机
驱动层（Driver）	外设抽象：OLED_ShowText(), Servo_SetAngle()
硬件抽象层（HAL）	底层接口：I2C_Write(), PWM_SetDuty(), ADC_Read()
硬件（Hardware）	MCU + 外设芯片

图 2: 软件分层架构

引导问题

- 1. 为什么要把软件分成不同的层次？如果应用层代码直接操作 I2C 寄存器，会带来什么问题？
- 2. 假设你验收时需要换一个不同型号的 OLED 屏幕（驱动芯片不同），分层架构下你只需要修改哪一层？
- 3. 对于这个桌宠项目，你会如何划分驱动层和应用层的边界？舵机动作序列的控制逻辑（比如“摆动 3 次、持续 2 秒”）应该放在哪一层？

4.2 程序调度方式

嵌入式系统的程序调度有多种方式，各有利弊。你需要根据项目的复杂度选择合适的方式。

4.2.1 方式一：超级循环（Super Loop）

```
1 void app_main() {
2     // 初始化
3     while (1) {
4         // 依次检查各个任务
5         check_button();
6         check_encoder();
7         check_light_sensor();
8         update_mood();
9         update_servo();
10        update_oled();
11        check_serial();
12        vTaskDelay(pdMS_TO_TICKS(10)); // 简单的延时
13    }
14 }
```

优点：简单直观，容易理解和实现。

缺点：所有任务串行执行，一个任务卡顿会影响所有其他任务；时序精度依赖于循环速度。

4.2.2 方式二：定时器中断 + 超级循环

将时间敏感的操作放在硬件定时器中断服务程序（ISR）中，而主循环处理非紧急任务。

引导问题

哪些操作适合放在 ISR 中？哪些不适合？为什么？（提示：ISR 的执行时间应该尽可能短）

4.2.3 方式三：RTOS 多任务

使用 FreeRTOS（ESP-IDF 默认集成）创建多个独立任务，由操作系统负责调度。

引导问题

1. 如果使用 RTOS，你会创建哪些任务？每个任务的职责是什么？
2. 不同任务之间如何共享数据（如心情值、动作状态）？多个任务同时修改同一个变量会有什么问题？
3. RTOS 虽然强大，但也会引入额外的复杂度（任务同步、优先级反转、栈溢出等）。对于基础功能而言，是否一定需要 RTOS？

重要

你需要在设计报告中的“架构设计”部分说明你选择的调度方式及其理由。

4.3 状态机设计

桌宠是一个典型的**状态驱动**系统——它的行为取决于当前所处的状态。

引导问题

1. 你的系统中存在哪些状态机？（提示：睡眠状态机、动作状态机、饱食状态机……）
2. 每个状态机有哪些状态？状态之间的转换条件是什么？（画出状态转换图）
3. 不同状态机之间如何互相影响？例如，进入睡眠状态会影响哪些其他状态机？

4.3.1 如何画状态转换图

状态转换图（State Transition Diagram）是描述状态机的标准方式，画法非常简单，只需要两种图形元素：

元素	画法	含义
状态	圆角矩形（或圆形），内部写上状态名称	系统或模块在某一时刻所处的“情况”
转换	带箭头的连线，旁边标注转移条件	当条件满足时，从一个状态跳转到另一个状态

此外，用一个实心圆点 + 箭头指向某个状态，表示“初始状态”（系统启动时进入的状态）。

下面以桌宠的睡眠状态机为例，图3展示了一个完整的状态转换图：

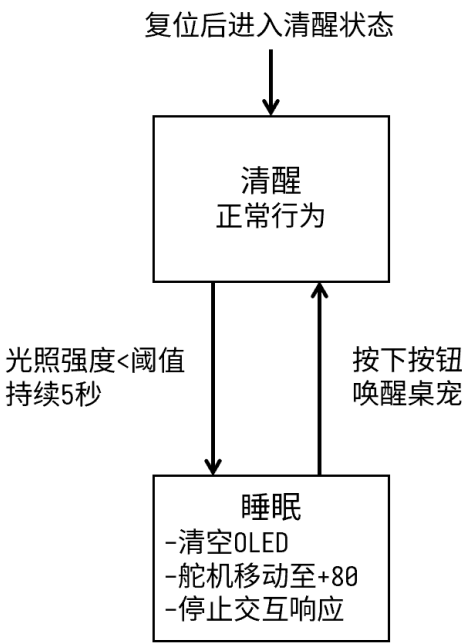


图 3: 状态转移图示例

绘制步骤：

1. 列出该子系统所有可能的状态
2. 确定初始状态
3. 思考每个状态下，什么事件/条件会导致状态切换
4. 画出所有状态和转换箭头

5. 检查：从每个状态出发，是否所有可能的转换都已画出？是否有“死状态”（进去了就出不来）？

重要

请为桌宠的每一个行为子系统画出状态转换图。这是设计报告中“架构设计”部分的重要内容。

4.4 模块间通信与耦合

模块之间的“耦合度”决定了你的系统是容易修改还是牵一发而动全身。

引导问题

1. “摸头”按钮被按下后，会影响哪些模块？（心情值？动作？OLED 显示？串口输出？）在你的设计中，这些影响是如何传递的——是按钮模块直接调用其他模块的函数，还是通过某种“事件”机制？
2. 如果老师要求把“摸头恢复 1 点心情值”改为“摸头恢复 2 点心情值”，你需要修改哪些代码？如果这个改动涉及 5 个文件，那可能意味着你的模块耦合度过高。
3. 考虑在软件中定义一个全局的“桌宠状态结构体”，各个模块通过读写这个结构体来交互。这种做法有什么优点和缺点？

4.5 定时与计时

大作业方案中有大量的时序约束（30 秒衰减、2 分钟饱食、10 秒冷却、5 秒光照判定等）。实现计时有几种不同的策略：

策略 A：硬件定时器中断计数 使用一个硬件定时器，以固定频率（如 100 Hz）产生中断，在 ISR 中维护一组软件计数器。

策略 B：RTOS 软件定时器 使用 FreeRTOS 的 Software Timer 功能。

策略 C：基于系统时钟的轮询 在主循环中不断读取系统时间（如 `xTaskGetTickCount()`），与上次记录的时间比较，判断是否到达触发条件。

引导问题

1. 上述三种策略各有什么适用场景？精度有什么区别？
2. 你的系统中同时存在多个需要计时的任务（心情衰减计数、饱食倒计时、冷却计时、光照计时、状态信息发送计时……），你打算如何组织这些计时器？
3. 睡眠状态下，哪些计时器应该暂停？哪些应该继续运行？

4.6 软件流程图：从思路到图形

软件流程图是设计报告中的另一张“必画图”。系统框图描述了“硬件怎么连”，流程图则描述“软件怎么跑”。

4.6.1 什么是软件流程图？

流程图（Flowchart）是一种用标准化的图形符号来描述程序执行顺序的图。它的核心价值在于：

- **理清逻辑**：画流程图的过程会迫使你一步步想清楚程序的执行顺序和分支条件。
- **便于交流**：图片比文字描述更直观。助教看流程图能快速理解你的程序结构。
- **辅助调试**：当程序运行不符合预期时，对照流程图检查，往往能快速定位是逻辑设计问题还是代码实现问题。

4.6.2 流程图的基本符号

流程图使用一套标准化的符号。对于嵌入式软件开发，掌握以下 6 种就足够了：

符号	名称	用途	示例
圆角矩形	开始/结束	标记程序的起点和终点	“程序开始”、“程序结束”
矩形	处理过程	执行某个操作或计算	“心情值 = 5”、“舵机转到 0°”
菱形	判断条件	根据条件分支	“按键是否按下？”
平行四边形	输入/输出	与外部设备交互	“读取 ADC 值”、“发送串口数据”
六边形（或双线矩形）	子程序调用	调用一个子函数	“执行舵机动作 ()”
带箭头连线	流程线	连接各步骤, 指示执行顺序	—

想一想

实际画图时，你不必严格区分“处理过程”和“输入/输出”——很多同学统一用矩形表示操作步骤，用菱形表示判断，用圆角矩形表示开始/结束，这就足够清晰了。关键在于**逻辑清楚**，而非符号刻板。

4.6.3 你需要画哪些流程图？

对于智能桌宠项目，你可能需要画以下几种流程图（至少画主程序流程）：

a) 主程序流程图 描述 `main()` 函数（或 `app_main()`）的执行流程。从系统上电开始，依次经过初始化 → 进入主循环 → 在主循环中轮询或响应事件。

图4 是一个**简化示意**（不是你的项目应该照搬的）：

b) 中断服务程序（ISR）流程图 如果你的设计中使用了外部中断（如按钮触发、编码器脉冲），你需要画出 ISR 的处理流程。ISR 流程图通常很短——记住 ISR 应该“快进快出”，只做必要的事（设置标志位、读取关键数据），复杂处理留给主循环。

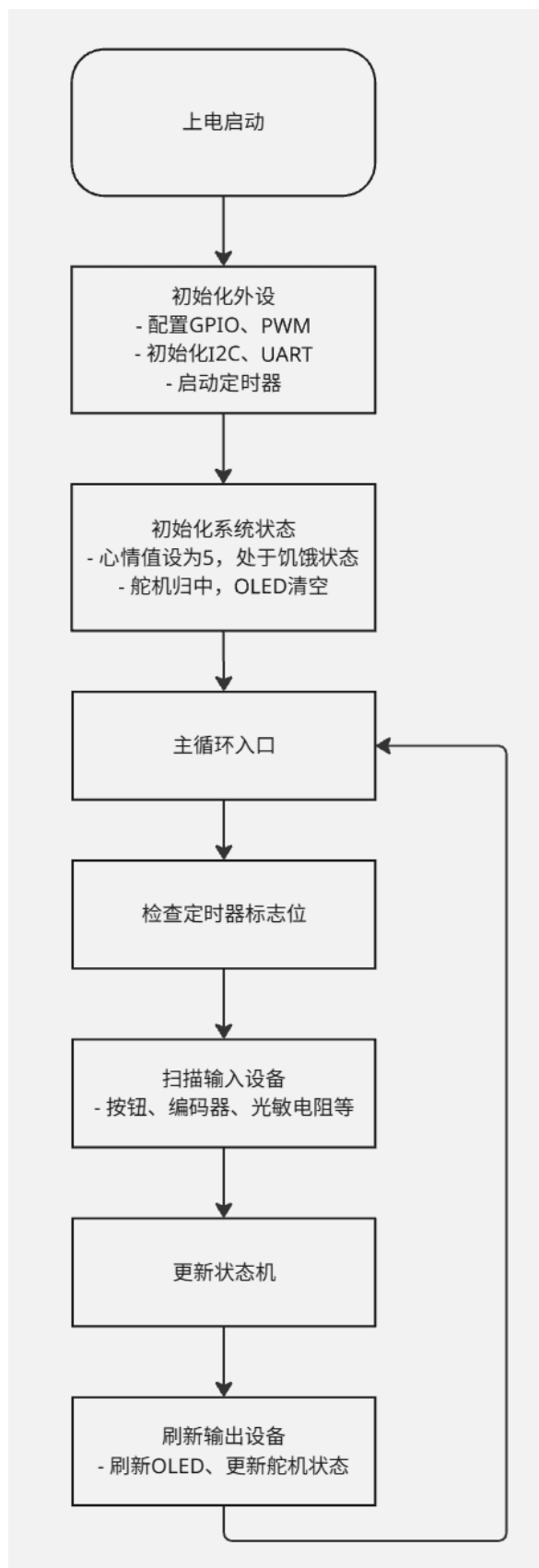


图 4: 主循环流程图示例

图5 为编码器 ISR 的示例：

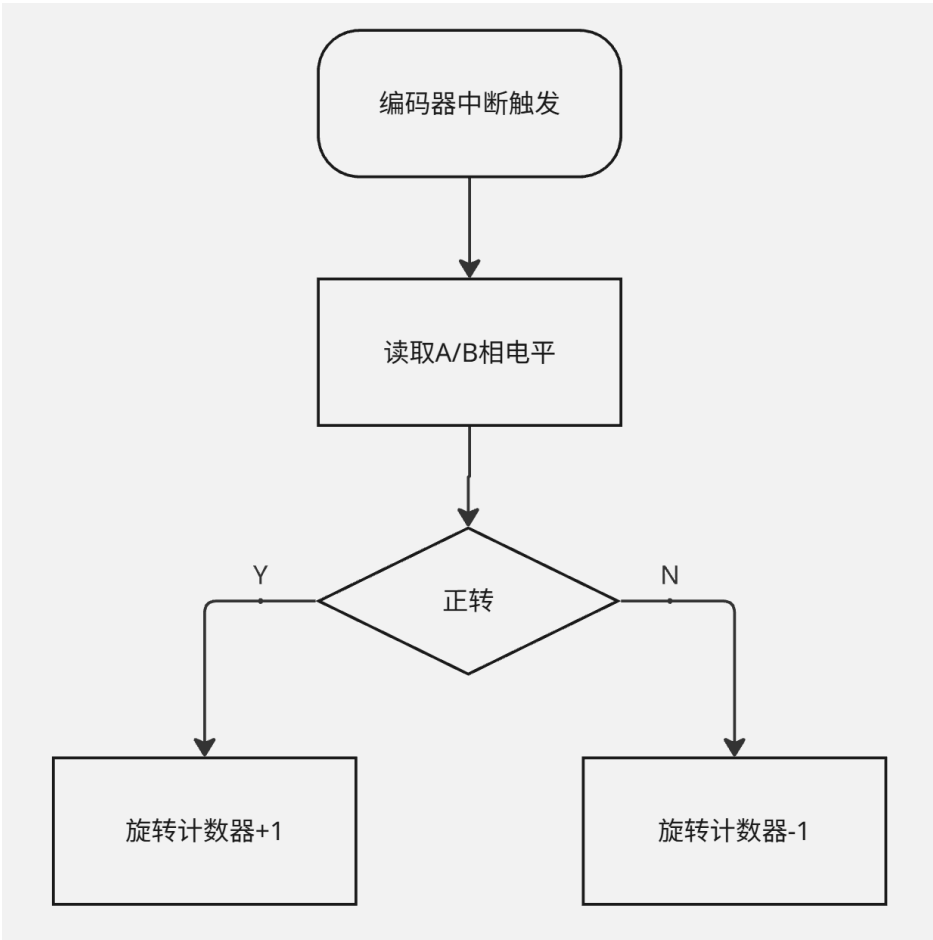


图 5: ISR 流程图示例

c) 子模块流程图 对于较复杂的子模块（如“自动动作触发判断”、“睡眠判定逻辑”），可以单独画流程图。设计报告中“功能实现”部分要求详细描述至少两个模块，为这两个模块配上子流程图会很有说服力。

4.6.4 流程图绘制步骤

第 1 步：确定范围

想清楚你要画的是哪一段逻辑的流程图。是主程序？某个子模块？还是一个 ISR？不要试图把所有东西挤在一张图里——几张清晰的小图远胜过一张混乱的大图。

第 2 步：列出关键节点

用自然语言列出这个流程中的所有关键操作和判断。例如对于主程序：

- 初始化
- 检查定时器
- 读按钮
- 读编码器
- 读光敏

- 判断是否触发摸头 → 是就执行， 否就跳过
- 判断是否触发喂食 → 是就执行， 否就跳过
- 判断是否进入睡眠
- 更新心情值
- 更新 OLED
-

第 3 步：确定顺序和分支

理清这些节点之间的先后关系。哪些是顺序执行的？哪些是有条件才执行的？有条件执行的节点，用菱形（判断）来分支。

第 4 步：落笔画图

按照从上到下、从左到右的顺序，把节点摆好，用箭头连接。判断节点的两条出口分别标注“是”和“否”（或“Y”和“N”）。

第 5 步：自检

画完后，用手指顺着流程线走一遍：“如果光照很低，程序走的路径是什么？”、“如果按键按下 + 正在睡觉，程序的路径是什么？” 确保每种情况下的路径都是正确且合理的。

4.6.5 画图工具

与系统框图类似，流程图同样推荐以下工具：

工具	特点	推荐度
纸 + 笔	最快打草稿	初稿首选
Draw.io	免费、在线、内置流程图符号库	强烈推荐
WPS/Word 流程图	常见办公软件内置功能	可用
PowerPoint	用形状 + 箭头手动拼	可用
Visio	专业流程图工具	可选

想一想

Draw.io 中新建文件后，左侧形状库搜索“Flowchart”即可找到所有标准流程图符号。

4.6.6 常见错误

- **流程线箭头方向混乱**：惯例是从上到下、从左到右。如果箭头往回指（循环），尽量从下方往回绕，不要横穿整个图。
- **判断框只有一条出口**：菱形判断框必须有至少两条出口（是/否），否则不需要判断。
- **一张图过于复杂**：如果一张图超过 15 个节点，考虑拆成多张图。主流程图可以画成概括性的（“更新系统状态”作为一个步骤），细节在子流程图中展开。
- **与代码实现不一致**：流程图应该反映代码的真实逻辑。如果在调试阶段修改了程序逻辑，记得同步更新流程图。
- **缺少初始化部分**：很多同学直接从主循环开始画，忘了系统启动后的初始化过程（外设配置、变量赋初值等），这部分也是程序逻辑的一部分。

- **流程图中途断开**：每一根流程线必须连接到某个节点，不能凭空消失。

4.6.7 引导问题

画出主程序流程图之后，请检查：

- 从“开始”出发，流程线能走到“结束”（或主循环的入口）吗？
- 每一个判断框的两个分支都有明确的去向吗？
- 你的 ISR 中的操作是否足够简短？有没有把耗时的操作（如更新 OLED）放在 ISR 中？
- 对照大作业方案逐条检查：方案中要求的每一种行为（摸头、喂食、睡眠、自动动作等），在你的流程图中是否都能找到对应的执行路径？

5 从设计到实现：下一步行动

当你完成了上述分析后，你应该已经拥有：

- ✓ 一份完整的功能需求和约束清单
- ✓ 一份外设接口类型分析
- ✓ 一份 ESP32-S3 引脚分配表（含分配理由）
- ✓ 一张系统硬件框图
- ✓ 各子系统的状态转换图
- ✓ 软件的模块划分和层次结构
- ✓ 一个主程序流程草图

有了这些“蓝图”，你就可以开始具体的实现了：

1. **搭建硬件原型**：用杜邦线和面包板连接各模块，先验证基本连通性。
2. **逐个模块验证**：从最简单的开始（如点亮 OLED、读取按钮、驱动一个舵机），确保每个模块独立工作正常。
3. **逐步集成**：将已验证的模块加入主程序，每次加入一个，确保不会引入新的 Bug。
4. **系统联调**：所有模块集成后，按照大作业方案逐条验证功能。

附录 附加资源

以下资源在你设计和实现过程中可能会用到：

- **ESP32-S3 技术参考手册**（Technical Reference Manual）：详细的外设说明和寄存器描述。
- **ESP32-S3-WROOM-1 数据手册**（Datasheet）：模组的引脚定义、电气特性、封装尺寸。
- **Arduino ESP32 编程指南**（Programming Guide）：官方的开发框架文档，包含大量示例代码。
- **你选修课的相关资料**：扩展功能的设计应建立在你选修课所学知识的基础上。

想一想

遇到困难时，先查阅数据手册和相关文档，再尝试独立调试。调试本身就是大作业的重要学习环节——你将在设计报告中的“调试与问题解决”部分记录你的 Debug 经历。

最后提醒：本文档提供的是一套**思考方法**，而不是一个**标准答案**。

不同同学设计出的系统架构可能完全不同，但只要逻辑清晰、功能完整、约束满足，都是好的设计。

大作业评分看重的是你的设计思路和实现过程，而不是是否采用了某种“最优方案”。